Feature matching using ORB algorithm in Python-OpenCV

Last Updated : 04 Jun, 2024

ORB is a fusion of FAST keypoint detector and BRIEF descriptor with some added features to improve the performance. **FAST** is **Features from Accelerated Segment Test** used to detect features from the provided image. It also uses a pyramid to produce multiscale-features. Now it doesn't compute the orientation and descriptors for the features, so this is where BRIEF comes in the role.

ORB uses BRIEF descriptors but as the BRIEF performs poorly with rotation. So what ORB does is to rotate the BRIEF according to the orientation of keypoints. Using the orientation of the patch, its rotation matrix is found and rotates the BRIEF to get the rotated version. ORB is an efficient alternative to SIFT or SURF algorithms used for feature extraction, in computation cost, matching performance, and mainly the patents. SIFT and SURF are patented and you are supposed to pay them for its use. But ORB is not patented.

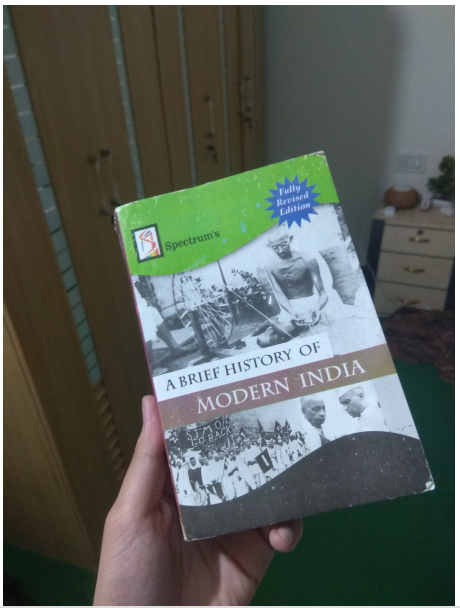
In this tutorial, we are going to learn how to find the features in an image and match them with the other images in a continuous Video.

Algorithm

- Take the query image and convert it to grayscale.
- Now Initialize the ORB detector and detect the keypoints in query image and scene.
- Compute the descriptors belonging to both the images.
- Match the keypoints using Brute Force Matcher.
- Show the matched images.

Below is the implementation.

Input image:




```python
import numpy as np
import cv2


# Read the query image as query_img
# and train image This query image
# is what you need to find in train image
# Save it in the same directory
# with the name image.jpg
query_img = cv2.imread('query.jpg')
train_img = cv2.imread('train.jpg')

# Convert it to grayscale
query_img_bw = cv2.cvtColor(query_img,cv2.COLOR_BGR2GRAY)
train_img_bw = cv2.cvtColor(train_img, cv2.COLOR_BGR2GRAY)

# Initialize the ORB detector algorithm
orb = cv2.ORB_create()

# Now detect the keypoints and compute
# the descriptors for the query image
# and train image
queryKeypoints, queryDescriptors =
orb.detectAndCompute(query_img_bw,None)
trainKeypoints, trainDescriptors =
orb.detectAndCompute(train_img_bw,None)

# Initialize the Matcher for matching
# the keypoints and then match the
# keypoints
matcher = cv2.BFMatcher()
matches = matcher.match(queryDescriptors,trainDescriptors)

# draw the matches to the final image
# containing both the images the drawMatches()
# function takes both images and keypoints
# and outputs the matched query image with
# its train image
```

```
final_img = cv2.drawMatches(query_img, queryKeypoints,
train_img, trainKeypoints, matches[:20],None)

final_img = cv2.resize(final_img, (1000,650))

# Show the final image
cv2.imshow("Matches", final_img)
cv2.waitKey(3000)
```

Output: